

TECHNICAL PROJECT REPORT - Yağız Selek

3D and Shader Programming · Game Programmer GP26

1. Project overview & final result



I created a procedural holographic force-field effect around a 3D sphere using the course OpenGL starter project with C++20, OpenGL 3.3 Core, and GLSL 3.30. The effect is driven primarily by the fragment shader, combining a view-dependent two-sided Fresnel rim with animated world-space scanlines and value-noise-based domain warping to create a dynamic holographic surface. The submitted version also includes two-sided transparent rendering, shader debug views for visualizing intermediate values, and Simple, Full, and Auto quality modes, with Auto using a distance-based LOD to skip the more expensive noise-warp calculations when the object is farther from the camera.

2. Technical implementation

The project starts with a sphere generated on the CPU. Each vertex contains a local-space position and an outward-facing normal. The vertex shader receives this data together with the model, view and projection matrices. It transforms the position into world space and then into clip space for rendering, while the normal is transformed into world space using the normal matrix. The world-space position and normal are then passed to the fragment shader after being interpolated across the triangle.

The fragment shader uses the interpolated world position and normal together with the camera position and time to create the hologram. It combines a view-dependent Fresnel rim with animated scanlines, energy bands and procedural distortion. In Full quality, the procedural coordinates are distorted using value-noise-based domain warping before some of the line patterns are evaluated. The shader finally outputs the hologram colour together with an alpha value for transparency.

Because the force field is transparent and should be visible from both sides, the sphere is drawn twice. The back surface is rendered first and the front surface second, using alpha blending with depth testing enabled but depth writes disabled. This allows both sides of the sphere to contribute to the final image.

3. Mathematics that controls the visible result

The visible result is mainly controlled by three mathematical ideas: a two-sided Fresnel-like rim, animated procedural scanlines, and value-noise-based domain warping.

a) Two-sided Fresnel-like rim

For the rim, I calculate a normalized world-space surface normal N and a normalized view direction V from the fragment towards the camera. Their dot product measures the angular alignment between the two directions:

$$F = 1 - |N \cdot V|$$

When N and V are almost parallel or anti-parallel, $|N \cdot V|$ is close to 1 and the rim contribution becomes small. At grazing angles they are close to perpendicular, so the dot product approaches 0 and the rim becomes strongest. The absolute value is important because the hologram is visible from both sides and I want the front and back surfaces to have the same angular response. I then shape the result using:

$$R = F^{2.4}$$

This concentrates the strongest values closer to the silhouette. It is a stylized Fresnel-like effect rather than a physically based Fresnel model. The resulting value contributes to both the brightness and alpha of the hologram, making the silhouette brighter and more visible than areas facing directly towards or away from the camera.

b) Animated procedural scanlines

The animated scanlines are generated procedurally from world-space coordinates and time. A sine function creates a repeating pattern whose frequency controls the spacing of the lines, while the time term changes its phase and makes the pattern move across the surface. The sine output is remapped from $[-1,1]$ into $[0,1]$ and then shaped using `smoothstep()`, so only the higher part of the

wave becomes a narrow bright band. Smaller sine-based offsets and hash-based breakup are also added to the line coordinates, preventing the final pattern from looking perfectly regular.

c) Value noise and domain warping

The most advanced mathematical part of the shader is the value-noise-based domain warp. The noise function uses `floor()` to find the current grid cell and `fract()` to find the local position inside it. A deterministic hash function generates pseudo-random values at the four corners of the cell. Instead of interpolating these values using the raw fractional coordinate, I first smooth the interpolation weight using the cubic Hermite function:

$$u = f^2(3 - 2f)$$

where f is the fractional position inside the grid cell, with the smoothing function applied independently to its X and Y components. The corner values are then interpolated first along one axis and then the other, producing smooth transitions between neighbouring grid cells. To animate the noise, the sampling coordinate is built from the world-space position, a scale factor and a time-dependent scrolling offset:

$$q = pS + td$$

where p is the original world-space XY coordinate, S controls the scale of the noise, t is time, and d is the scrolling vector that controls its direction and speed. I sample the resulting noise field twice using an offset between the two samples. The two scalar values are remapped from $[0,1]$ to $[-1,1]$ and used as the X and Y components of a 2D displacement vector. The coordinate warp can then be described as:

$$p' = p + A w(p, t)$$

where p is the original procedural coordinate, $w(p,t)$ is the animated noise displacement and A is the warp strength.

The same operation appears directly in the shader:

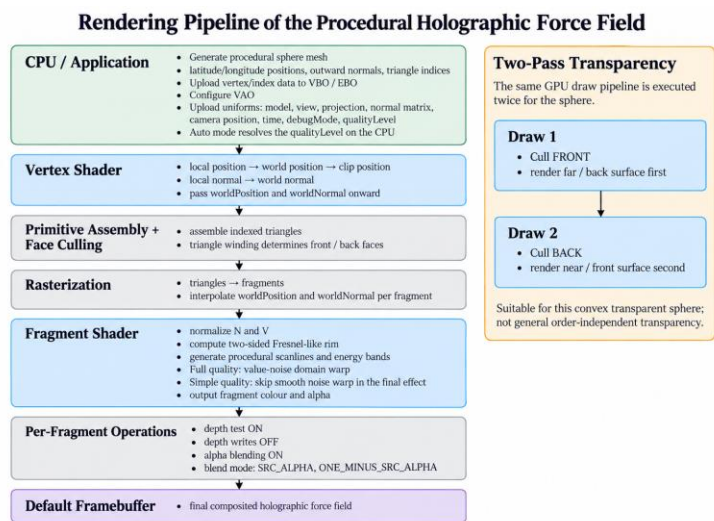
```
vec2 noiseWarp = vec2(noiseX, noiseY) * 2.0 - 1.0;
warpedPos = pos + noiseWarp * WarpStrength;
```

Here, the two noise samples are first remapped to a signed 2D displacement and then added to the original coordinate using `WarpStrength`.

The important part is that the noise is not simply added to the final colour. Instead, it changes the coordinates before parts of the procedural pattern are evaluated. The main scanlines use the warped coordinates as their base domain, while the energy bands blend between the original and warped coordinates so that the different layers do not deform in exactly the same way.

The hash-based breakup remains based on the original coordinate domain. This adds a smoother and more organic deformation on top of the existing sine- and hash-based instability. I can also isolate this visually using the debug views, comparing the scanlines before and after the noise warp.

4. Rendering pipeline illustration



AI-generated rendering pipeline illustration for the procedural holographic force field project.

Green highlights the CPU/application-side work, blue highlights the shader stages, yellow highlights the project-specific two-pass transparency process, grey represents fixed-function OpenGL pipeline stages, and purple marks the final framebuffer output.

Own work is shown in green, blue, and yellow; grey boxes represent fixed-function OpenGL stages, and purple indicates the final framebuffer output.

The sphere mesh is generated on the CPU and sent to the GPU together with the transformation matrices, camera position, time, debug mode, and resolved quality level.

The vertex shader transforms the geometry into world and clip space, after which triangle winding is used for front/back face classification and rasterization interpolates the world-space position and normal for each fragment. The fragment shader then computes the Fresnel-like rim, procedural scanlines and energy bands, with value-noise domain warping enabled in Full quality. Because the force field is transparent and visible from both sides, the GPU draw pipeline is executed twice: the far/back surface is rendered first and the near/front surface second, with depth testing enabled, depth writes disabled, and standard alpha blending enabled.

5. Lighting / visual-effect reasoning & wider rendering concept

Traditional scene lighting is not central to this effect. The force field is designed to have a self-illuminated, emissive-looking appearance rather than being lit by a conventional diffuse or specular lighting model. Its appearance is mainly driven by the world-space surface normal, view direction, procedural animation and transparency. The two-sided Fresnel-like term increases the field intensity at grazing viewing angles, making the silhouette brighter, while the Fresnel-derived intensity and a slower pulse also affect alpha before the result is composited using standard alpha blending.

A relevant wider rendering concept is HDR rendering, although it is not implemented in the submitted project. The current shader can mathematically output RGB components above 1.0, but the renderer does not use a floating-point HDR render target and renders directly to the default framebuffer. In a modern HDR pipeline, the force field could instead be rendered to a floating-point target such as RGBA16F, preserving these bright values for later processing. A separate bloom post-process could then extract and blur bright regions before they are combined with the scene and tone-mapped for display. This could strengthen the hologram's glowing appearance without changing the core Fresnel, scanline and domain-warping logic.

6. Optimization: what did you change and what did it cost/save?

I targeted the cost of the value-noise domain warping in the fragment shader. In Full quality, two 2D value-noise samples are calculated for each fragment and used to distort the coordinates of the procedural patterns. In Simple quality, this smooth noise warp is skipped. The Fresnel rim, scanlines, energy bands, hash-based breakup, animation and transparency are still evaluated, so the main holographic effect remains visible while the more expensive procedural distortion is removed.

Question	Before - Full	After - Simple	Trade-off / result
What workload did I target?	Two 2D value-noise samples are evaluated per fragment to create the domain-warp displacement.	Simple quality skips the value-noise domain warp. Auto selects between Full and Simple based on camera distance.	Less fragment-shader work, but the smooth noise-based distortion is lost in Simple quality.
How did I measure?	Eight Full benchmark runs, each using 600 GPU timing samples. Mean 5% trimmed GPU time: 0.03592 ms.	Eight Simple benchmark runs under the same benchmark setup. Mean 5% trimmed GPU time: 0.02044 ms.	Simple reduced the measured hologram GPU draw time by approximately 43.1%.

The Auto mode uses the distance between the camera and the sphere centre to choose the quality level. It switches to Full below a distance of 5.0 and to Simple above 6.0. Between those two distances it keeps the previous quality level, creating a small hysteresis range so the shader does not rapidly switch back and forth around a single threshold.

For the performance test, I alternated between Simple and Full for eight benchmark pairs on an NVIDIA GeForce RTX 4070 Laptop GPU using OpenGL 3.3. Each benchmark used a 120-frame warm-up followed by 600 GPU timing samples. The benchmark resets the camera to the same position and disables VSync while measuring, and the GPU timer covers the same two transparent sphere draw calls in both modes.

The absolute timings changed slightly between runs, which is expected on a laptop GPU, but Simple was faster in all eight paired tests. The measured reduction ranged from approximately 39.9% to 47.2%, with an overall reduction of 43.1% based on the averaged trimmed-mean results. The optimization therefore trades some of the smooth domain-warp detail for a consistent reduction in the GPU time spent drawing the hologram.

7. Development, debugging & limitations

a) Domain-warp debugging

After adding value-noise domain warping, it was difficult to tell whether the noise was actually changing the scanline coordinates or whether the extra movement was coming from the other procedural terms. My hypothesis was that comparing the same pattern before and after the coordinate warp would isolate its effect. I added debug views for an unwarped scanline, a noise-warped scanline, and the final effect with the smooth warp disabled. This showed that the warp was working, but also revealed that my “unwarped” path was not completely unwarped: some calculations still used warpedPos directly instead of the selectable effectPos. I changed the relevant jitter, main-line and energy-band calculations to use effectPos, which made the comparison clean and confirmed that the noise was deforming the coordinate domain rather than simply changing the final colour.

b) Two-sided transparency and face culling

When I first set up the sphere to render the back surface before the front surface, the culling behaviour did not match what I expected even though the vertex normals pointed outward. I initially suspected the normals, so I compared the two culling passes and checked the generated triangle indices. The problem was the triangle winding: `glCullFace` uses winding to classify front and back faces, while the normals are only used by the shader for calculations such as the Fresnel term. I corrected the index order and kept the intended two-pass setup: cull front faces to draw the far side first, then cull back faces to draw the near side. This separated two concepts that I had initially treated as related: surface normals control shading, while triangle winding controls face culling.

c) Limitations

The main limitation is that this transparency method only works reliably for the simple convex sphere used in the project. It is not a general order-independent transparency solution, so concave meshes, intersecting transparent surfaces or several overlapping transparent objects could still blend in the wrong order. I also decided not to add a custom HDR/bloom pipeline once the core shader was stable, and instead kept HDR as a wider rendering concept in the report. This kept the scope focused on the domain warping, transparency and measured shader LOD rather than adding another unfinished system.

8. Conclusion

I ended up with a procedural holographic force field that combines a two-sided Fresnel-like rim, animated scanlines, transparency and value-noise-based domain warping. The part I am most satisfied with technically is the domain warping, because the noise changes the coordinates of the procedural patterns instead of simply being added to the final colour. I also learned a lot about how to approach something I had never built before, understand the techniques quickly, and create the effect from scratch step by step. If I had more time, I would make the force field react to mouse input so the distortion could respond interactively to the user.

9. Credits, references & AI-assisted work

- Futuregames 3D and Shader Programming starter project: used as the base OpenGL project and extended for my own shader, rendering and benchmarking work.
- Course feedback and code review: helped me identify issues such as the sphere winding/culling behaviour and improve the final rendering setup.
- AI: I used it mainly to explain OpenGL/GLSL concepts, help me debug problems, review parts of my code, and discuss benchmarking and report structure. I did not use it as a replacement for implementing the project: I tested, changed and integrated the final code myself and made sure I understood the techniques I submitted.